

Project Documentation: Round-Robin Load Balancing with Kubernetes, Docker and Linux

Submitted by: **Pranav Mehta** (Student, Chandigarh University, UID: 21BCS11480)

GitHub repository link: <https://github.com/PranavMehta15/Kubernetes-Load-Balancing-Demo>

Project Overview

This project involves setting up a round-robin load balancing system using Kubernetes. The system includes two servers (server1 and server2) and a client that interacts with these servers. The client performs requests to both servers, and Kubernetes manages the load balancing and fault tolerance.

Architecture

- **Servers:** Two server deployments (server1 and server2) that serve HTTP requests.
- **Client:** A client deployment that sends HTTP requests to both servers.
- **Load Balancing:** Managed by Kubernetes services that distribute traffic between server pods.
- **Fault Tolerance:** Handled by Kubernetes, which automatically restarts failed pods.

Setup and Environment

Virtual Machine:

- Platform: Ubuntu
- Version: Ubuntu 24.04 LTS
- Virtualization: VirtualBox

Methodology

1. Setting Up the Environment

Install MicroK8s

MicroK8s was installed as a lightweight Kubernetes distribution:

```
sudo snap install microk8s --classic
```

Verify installation:

microk8s kubectl version

Enable Necessary Add-ons

Enabled essential add-ons for Kubernetes:

microk8s enable dns

microk8s enable storage

Project Files Overview

The project consists of several files for defining Kubernetes resources, Docker configurations, and application logic. Below is a summary of the files and their purposes:

1. Kubernetes Configuration Files:

- client-deployment.yaml
- server1-deployment.yaml
- server2-deployment.yaml
- client-service.yaml
- server1-service.yaml
- server2-service.yaml
- network-policy.yaml

2. Docker Configuration Files:

- Dockerfile (Client)
- Dockerfile (Server 1)
- Dockerfile (Server 2)

3. Application Files:

- client.py
- server1.py
- server2.py

Deploy the client:

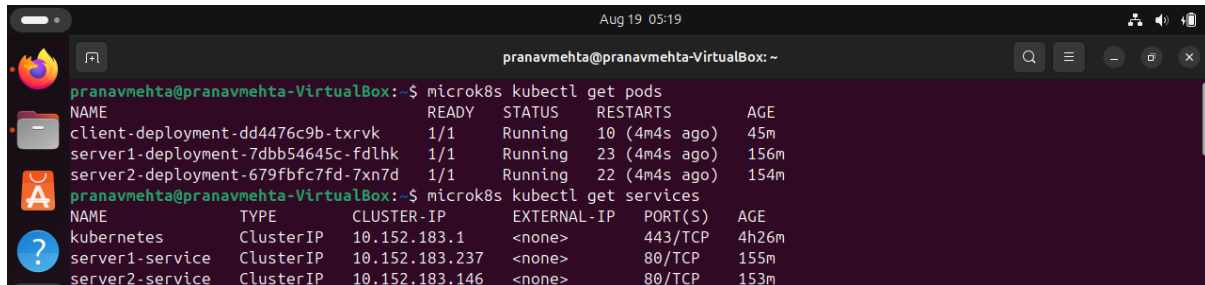
microk8s kubectl apply -f client-deployment.yaml

4. Testing and Verification

Verify Pods and Services

Check the status of pods:

microk8s kubectl get pods

A terminal window titled 'pranavmehta@pranavmehta-VirtualBox: ~' showing the output of 'microk8s kubectl get pods' and 'microk8s kubectl get services'. The pod list shows three running pods: client-deployment, server1-deployment, and server2-deployment. The service list shows three services: kubernetes, server1-service, and server2-service.

```
pranavmehta@pranavmehta-VirtualBox:~$ microk8s kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
client-deployment-dd4476c9b-txrvk   1/1     Running   10 (4m4s ago)  45m
server1-deployment-7dbb54645c-fdlhk 1/1     Running   23 (4m4s ago)  156m
server2-deployment-679fbfc7fd-7xn7d 1/1     Running   22 (4m4s ago)  154m
pranavmehta@pranavmehta-VirtualBox:~$ microk8s kubectl get services
NAME             TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes       ClusterIP   10.152.183.1 <none>        443/TCP    4h26m
server1-service   ClusterIP   10.152.183.237 <none>        80/TCP     155m
server2-service   ClusterIP   10.152.183.146 <none>        80/TCP     153m
```

Image (i): Client server, Server1 & Server2 are up and running

Check the services:

microk8s kubectl get services

Simulate Failures

To demonstrate fault tolerance, delete a server pod:

microk8s kubectl delete pod <server1-pod-name>

Observe Kubernetes automatically restarting the pod.

A terminal window showing the deletion of a pod and its subsequent restart. The first command deletes the pod 'server1-deployment-7dbb54645c-fdlhk'. The second 'get pods' command shows the pod has been replaced by 'server1-deployment-7dbb54645c-wprv5'. The third 'get pods' command shows the pod is still running.

```
pranavmehta@pranavmehta-VirtualBox:~$ microk8s kubectl delete pod server1-deployment-7dbb54645c-fdlhk
pod "server1-deployment-7dbb54645c-fdlhk" deleted
microk8s kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
client-deployment-dd4476c9b-txrvk   1/1     Running   10 (10m ago)  51m
server1-deployment-7dbb54645c-wprv5 1/1     Running   0           32s
server2-deployment-679fbfc7fd-7xn7d 1/1     Running   22 (10m ago)  160m
pranavmehta@pranavmehta-VirtualBox:~$ microk8s kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
client-deployment-dd4476c9b-txrvk   1/1     Running   10 (12m ago)  53m
server1-deployment-7dbb54645c-wprv5 1/1     Running   0           2m49s
server2-deployment-679fbfc7fd-7xn7d 1/1     Running   22 (12m ago)  162m
pranavmehta@pranavmehta-VirtualBox:~$
```

Image (ii): Deleting server1 to simulate a crash, resulting in Kubernetes automatically starting it again

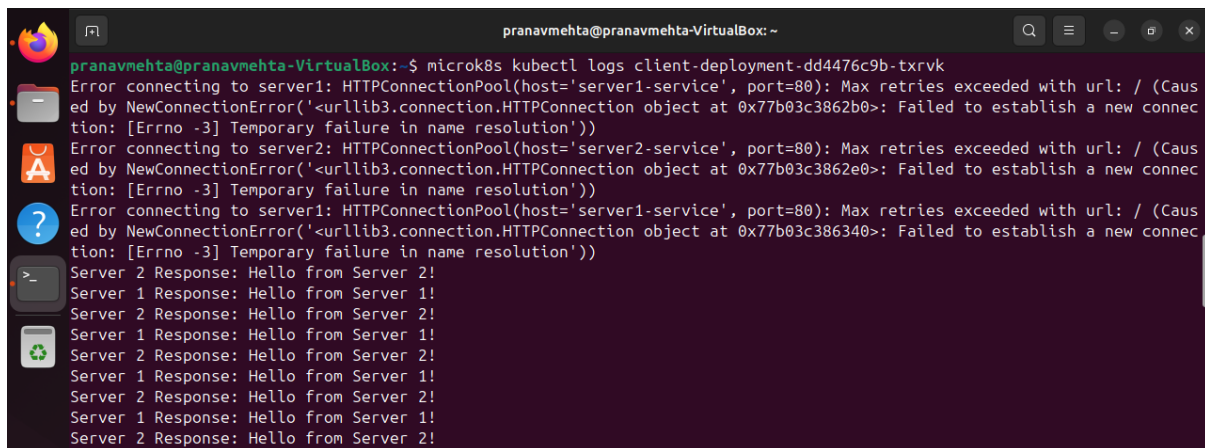
Verify Load Balancing

Connect to the client pod and ensure it can communicate with both servers:

microk8s kubectl exec -it <client-pod-name> -- /bin/sh

curl <http://server1-service>

curl <http://server2-service>



```
pranavmehta@pranavmehta-VirtualBox: ~  
pranavmehta@pranavmehta-VirtualBox:~$ microk8s kubectl logs client-deployment-dd4476c9b-txrvk  
Error connecting to server1: HTTPConnectionPool(host='server1-service', port=80): Max retries exceeded with url: / (Caus  
ed by NewConnectionError('<urllib3.connection.HTTPConnection object at 0x77b03c3862b0>: Failed to establish a new connec  
tion: [Errno -3] Temporary failure in name resolution'))  
Error connecting to server2: HTTPConnectionPool(host='server2-service', port=80): Max retries exceeded with url: / (Caus  
ed by NewConnectionError('<urllib3.connection.HTTPConnection object at 0x77b03c3862e0>: Failed to establish a new connec  
tion: [Errno -3] Temporary failure in name resolution'))  
Error connecting to server1: HTTPConnectionPool(host='server1-service', port=80): Max retries exceeded with url: / (Caus  
ed by NewConnectionError('<urllib3.connection.HTTPConnection object at 0x77b03c386340>: Failed to establish a new connec  
tion: [Errno -3] Temporary failure in name resolution'))  
Server 2 Response: Hello from Server 2!  
Server 1 Response: Hello from Server 1!  
Server 2 Response: Hello from Server 2!  
Server 1 Response: Hello from Server 1!  
Server 2 Response: Hello from Server 2!  
Server 1 Response: Hello from Server 1!  
Server 2 Response: Hello from Server 2!  
Server 1 Response: Hello from Server 1!  
Server 2 Response: Hello from Server 2!  
Server 1 Response: Hello from Server 1!  
Server 2 Response: Hello from Server 2!
```

Image (iii): Client server receives responses from both servers creating a loop as it follows round robin algorithm

Conclusion

The project demonstrates the use of Kubernetes for load balancing and fault tolerance in a microservices environment. The system effectively balances HTTP requests between multiple server instances and automatically handles server failures.

Commands Summary

- **Install MicroK8s:** `sudo snap install microk8s --classic`
- **Enable Add-ons:** `microk8s enable dns, microk8s enable storage`
- **Deploy Server 1:** `microk8s kubectl apply -f server1-deployment.yaml`
- **Create Server 1 Service:** `microk8s kubectl apply -f server1-service.yaml`
- **Deploy Client:** `microk8s kubectl apply -f client-deployment.yaml`
- **Check Pods:** `microk8s kubectl get pods`
- **Check Services:** `microk8s kubectl get services`
- **Simulate Failure:** `microk8s kubectl delete pod <server1-pod-name>`
- **Scale Deployments:** `microk8s kubectl scale deployment/server1-deployment --replicas=3`